Title & Team Details

Converting Legacy Code into Modern Framework Automatically

Team Member

B. Geethanjali (2303A51005)

G.Haripriya (2303A51104)

Branch

CSE

Guide

Naresh Kumar Sripada

Introduction

The Challenge

Legacy systems are difficult to maintain, scale, and update due to outdated technologies and tightly coupled architectures. Manual migration is time-consuming, costly, and prone to human errors.

Many organizations face challenges in adapting legacy applications to modern platforms like cloud and microservices.

This project proposes an automated solution that leverages AI and rule-based techniques to analyze, refactor, and convert legacy code into modern frameworks — improving efficiency, reducing risks, and enabling faster, reliable software modernization.

Key Points

1. Legacy systems are hard to maintain & scale
2. Outdated technologies and architectures
3. Manual migration is slow and error-prone
4. Hard to adapt to cloud & modern frameworks
5. Need for automation in modernization
6. Uses AI + rule-based techniques
7. Improves efficiency and reduces errors

Objectives

What we aim to achieve

01

Automate

Legacy code conversion end-to-end

02

Reduce Effort

Minimize manual work and human errors

03

Improve Performance

Boost scalability of applications

04

Cloud-Ready

Enable modern cloud architectures

05

Multi-Language

Support varied legacy systems

06

AI Accuracy

Reliable code transformation

07

Low Downtime

Minimize migration disruption

08

Quality

Enhance code maintainability

09

Faster Delivery

Speed up dev & deployment

10

CI/CD Ready

Integrate with automation pipelines

Stakeholders

Industry & Enterprises

- Software Companies
- IT Service Providers
- Enterprises with legacy systems
- Startups modernizing apps

Technical Teams

- Developers & DevOps Teams
- System Architects
- Maintenance & Support Teams
- Project Managers

Platforms & Sectors

- Cloud Service Providers
- Government Sector
- Banking & Financial Services
- Public Infrastructure

Tools & Technologies

Languages	Java • Python • JavaScript
Frameworks	Spring Boot • Node.js • Express
Code Analysis	Static Analysis • AST Parsers
CI / CD	GitHub Actions • Jenkins
Database	MySQL • MongoDB
Tools	VS Code • GitHub • Docker
AI / ML	ML Models • NLP Techniques
Transformation	Rule Engines • Refactoring Tools
Cloud	AWS • Azure • Google Cloud
Version Control	Git

Prompts Used

Patent-based AI instructions

Prompt 01

Analyze legacy code structure and dependencies

Prompt 02

Convert legacy code into modern framework

Prompt 03

Perform automated code refactoring

Prompt 04

Generate optimized modern code

Prompt 05

Apply AI-based semantic transformation

Prompt 06

Validate and test transformed code

Prompt 07

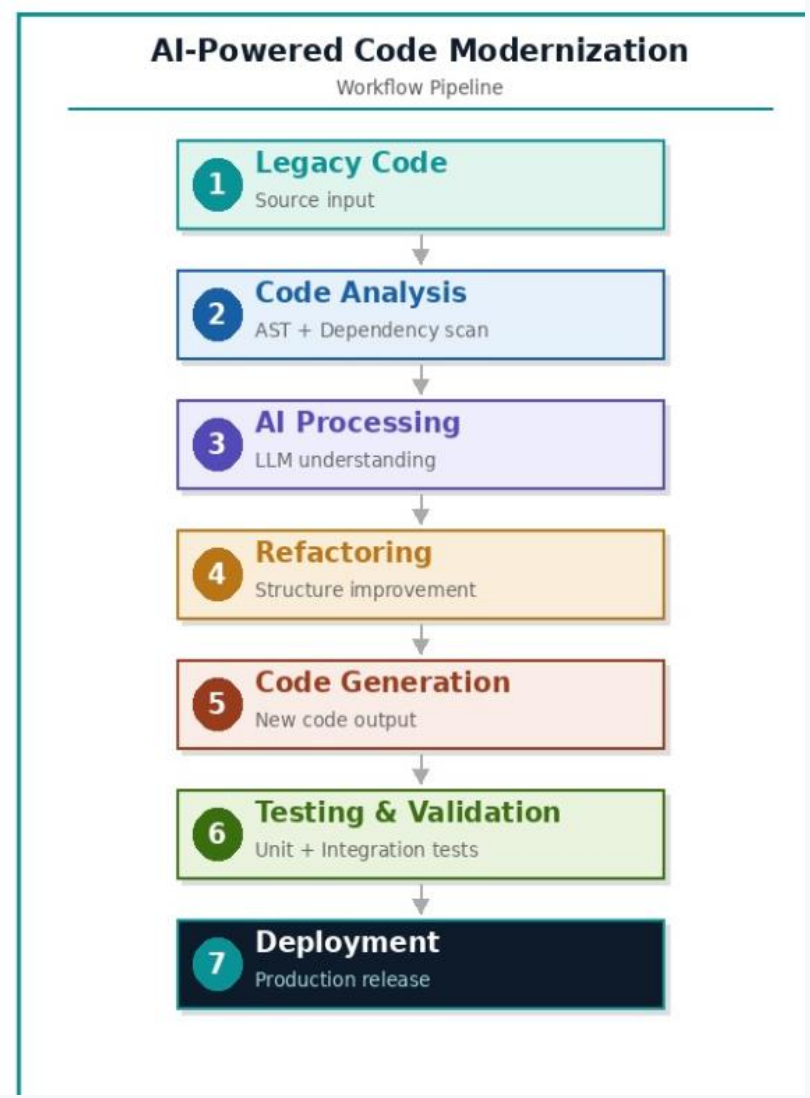
Enable CI/CD-based deployment

Prompt 08

Optimize code performance and scalability

Workflow

End-to-end modernization pipeline



Pipeline Explanation

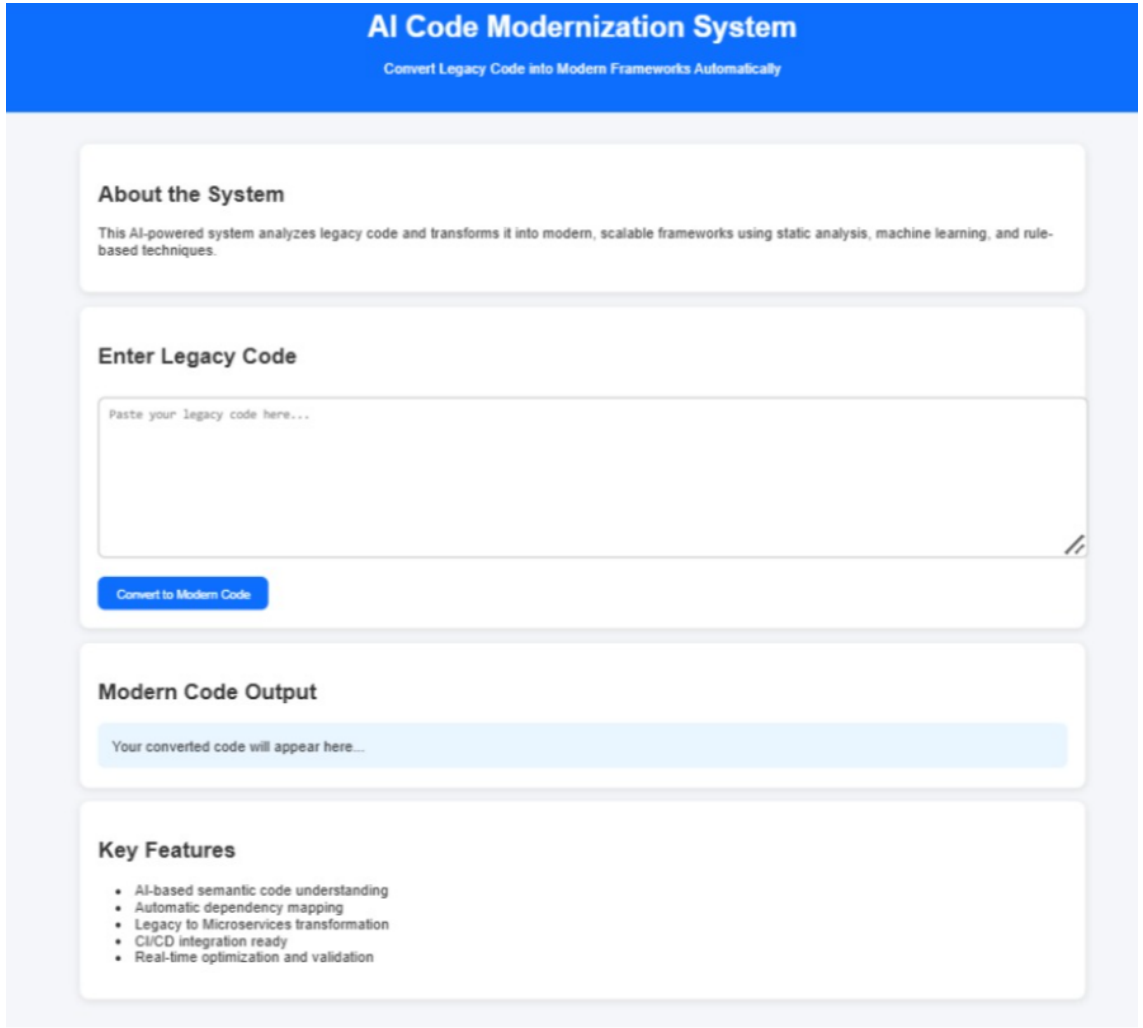
1. **Legacy Code Input** Source code is fed into the system.
2. **Code Analysis** AST + dependency scan to map structure.
3. **AI Processing** LLM understands semantics and intent.
4. **Refactoring** Outdated patterns are restructured.
5. **Code Generation** Optimized modern code is produced.
6. **Testing & Validation** Unit + integration tests verify behavior.
7. **Deployment** Released via modern CI/CD environments.

Front End View

User interface of the modernization system

Interface Highlights

Header	Branded hero with system title and subtitle
About	Brief description of the AI-powered system
Code Input	Paste legacy code into a clean textarea
Convert Action	Single-click conversion to modern code
Output Panel	Modernized code displayed clearly
Key Features	Highlights of the platform capabilities



Prompts & Code

PROMPT 1

Create a responsive single-page web interface for an AI-based system that converts legacy code into modern frameworks.

PROMPT 2

Implement a JavaScript-based simulation of AI code transformation with rule-based replacements and a loading effect.

AI OUTPUT

A clean, responsive single-page website with modern UI (gradient + glass effect), header / hero section, about section, code input + output area, and full mobile + desktop responsiveness — generated and refined through iterative prompts.

Generated Code

💻 Full Working Code (Paste into `.html`)

HTML

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>AI Code Converter</title>

<style>
    body {
        margin: 0;
        font-family: Arial, sans-serif;
        background: linear-gradient(135deg, #1e3c72, #2a5298);
        color: white;
    }

    header {
        text-align: center;
        padding: 20px;
        background: rgba(0,0,0,0.3);
    }

    section {
        max-width: 1000px;
        margin: auto;
        padding: 20px;
    }

    .container {
```

References

01	**Spring Boot**	Official Documentation — backend framework
02	**Node.js**	Official Documentation — JavaScript runtime
03	**GitHub**	Project repository & code references
04	**Online Tutorials**	Resources on legacy code conversion
05	**AI Tools Docs**	Used for code generation and refactoring

THANK YOU

B. Geethanjali • 2303A51005 • CSE

Guide: Naresh Kumar Sripada